

Sujet de Projet : "E-Shop Monitor" – Observabilité & Optimisation du Tunnel d'Achat

1. Contexte du Projet

Vous êtes Lead Développeur Web au sein d'une startup e-commerce en plein essor : "**Eco-Hardware**". Le site subit actuellement des pertes de chiffre d'affaires inexpliquées. Le Product Owner soupçonne des bugs intermittents lors du paiement et un taux d'abandon anormalement élevé dans le tunnel d'achat.

Votre mission est de déployer une infrastructure d'observation **100% open source et auto-hébergée**, de l'intégrer à l'application web, et de fournir un tableau de bord analytique pour optimiser l'expérience utilisateur et technique.

2. Objectifs Pédagogiques

- Maîtriser le déploiement d'une stack de monitoring multi-services avec **Docker Compose**.
- Savoir configurer et segmenter l'analytique produit sans cookies (**Umami**, **Plausible** ou autre voir selfh.st/apps).
- Savoir intercepter, filtrer et centraliser les erreurs applicatives (**GlitchTip**).
- Mettre en place un plan de marquage (événements personnalisés) pour suivre un tunnel de conversion.

3. Architecture Technique Requise

L'ensemble de l'environnement doit être orchestré via un ou plusieurs fichiers `compose.yml` et tous les services conteneurisés de la façon suivante :

- Application Web
 - Service Node (Nuxt.js, Next.js)
 - Service Nginx, Apache ou Traefik (recommandé)
- Glitchtip
 - 1 base de données PostgreSQL

- 1 base de données Redis
- 1 service Glitchtip
- Analytique
 - 1 base de données PostgreSQL
 - 1 service Umami, Plausible ou autre

4. Travail à Réaliser (Cahier des Charges)

Partie 1 : Infrastructure Docker (Obligatoire)

Vous devez fournir une configuration Docker Compose comprenant :

1. **L'Application Web** : Une application (idéalement Vue.js 3, Nuxt ou React) simulant un site e-commerce simple (Page Accueil, Liste produits, Détail produit, Panier, Paiement).
2. **GlitchTip** : Avec sa base de données PostgreSQL dédiée et son instance Redis pour la gestion des tâches de fond.
3. **Umami** : Connecté à sa propre base de données pour stocker les métriques analytiques.

Partie 2 : Implémentation de la Télémétrie (GlitchTip)

- **Capture des erreurs frontend** : Intégrer le SDK pour capturer automatiquement toutes les exceptions JavaScript non gérées.
- **Simulation de pannes** : Créer volontairement un bouton de paiement défaillant (ex: qui génère un `TypeError` ou une promesse rejetée une fois sur trois) pour valider la remontée d'alertes dans GlitchTip.
- **Suivi de performance (Performance Tracking)** : Configurer le SDK pour mesurer le temps de chargement des composants clés (notamment la page de validation de commande).

Partie 3 : Implémentation de l'Analytique & Tunnel de Conversion (Umami)

Vous devez configurer le tracker Umami pour remonter les indicateurs standards et spécifiques demandés par le métier.

A. Métriques Standards (Gérées nativement par Umami)

- **Nombre de visites** (Sessions uniques) et pages vues.
- **Durée moyenne de session** (pour identifier l'engagement).
- **Taux de rebond** (Bounce Rate) sur la page d'accueil.

B. Suivi du Tunnel d'Achat (Événements Personnalisés / Custom Events)

Vous devez implémenter un plan de marquage pour suivre pas à pas le tunnel de conversion suivant :

1. `view_product` : L'utilisateur consulte une fiche produit.
2. `add_to_cart` : L'utilisateur ajoute un produit au panier.
3. `checkout_start` : L'utilisateur clique sur "Passer la commande" et accède au formulaire de livraison/paiement.
4. `checkout_success` : L'utilisateur a payé avec succès (Arrivée sur la page de confirmation).

C. Métriques Métier Avancées (À tracker via des propriétés d'événements)

- **Taux de conversion global du tunnel** : (Nombre de `checkout_success` / Nombre de visites totales).
- **Taux d'abandon par étape** : Identifier l'étape exacte du tunnel où les utilisateurs s'en vont le plus (ex: abandon entre le panier et le paiement).
- **Métriques additionnelles (liste non-exhaustive)** :
 - *Montant du panier moyen* (Passer le prix en propriété de l'événement `checkout_success`).
 - *Suivi de l'origine du trafic* (via les paramètres URL `ref` ou `utm_source`) pour les campagne de pub.

5. Livrables Attendus

Le rendu du projet se fera sous la forme d'un dépôt Git contenant :

1. **Le code source de l'application web** et les fichiers de configuration Docker (`compose.yml` , `.env.example`).
2. **Un fichier `README.md` explicatif** détaillant la procédure pour lancer toute la stack en une seule commande (`docker compose up`).

3. Un rapport d'observabilité (PDF ou Markdown) contenant :

- Des captures d'écran de votre dashboard Umami montrant le tunnel d'achat complété (après avoir simulé plusieurs parcours utilisateurs).
- Une analyse du taux de rebond et du taux de conversion observés lors de vos tests.
- Une capture d'écran de GlitchTip montrant la stack trace de l'erreur simulée lors du paiement défaillant, avec une brève explication de la manière dont un développeur peut la résoudre grâce à ces informations.

6. Critères d'Évaluation

Hors malus (failles de sécurité, mauvaises pratique de code, etc...).

Critère	Description	Points
Infrastructure Docker	L'ensemble des services démarre sans erreur via Docker Compose, volumes persistants configurés.	/5
Intégration Télémétrie	GlitchTip capture correctement les erreurs et fournit les détails (OS, Navigateur, Stacktrace).	/4
Analytique de Base	Umami est fonctionnel et remonte le trafic global, le taux de rebond et le temps de session.	/3
Tunnel d'Achat	Les 4 étapes du tunnel de conversion déclenchent des événements personnalisés précis.	/5
Qualité du code & Rapport	Code propre, respect RGPD (pas de PII dans les logs), clarté du rapport final.	/3
Total		/20